Parametry, argumenty i wartości domyślne

Parametr (inaczej *parametr formalny*) — zmienna będąca częścią nagłówka funkcji. Np. poniższa funkcja `f` ma trzy parametry: `a`, `b` i `c`.

In [2]:
```
def f(a, b, c):
    return a * b + c
```

Argument (inaczej *parametr aktualny*) — wyrażenie przekazane podczas wywołania funkcji. Np. w poniższym wywołaniu argumentami są `x`, `3.4` i `y + 2.3`.

In [3]:
```
x = 3.2
y = -4.5
```

In [4]:
```
f(x, 3.4, y + 2.3)
```

Out[4]:
```
8.68
```

Mówimy, że funkcja **ma** parametry i **przyjmuje** argumenty.

Przekazywanie argumentów do funkcji w Pythonie

Przypomnijmy sposób przekazywania argumentów. Podczas wywołania parametr staje się lokalną nazwą wskazującą przekazany obiekt. Przypisanie do parametru zmienia lokalne powiązanie, a nie zmienną używaną przez wywołujący kod. Zmiana mutowalnego obiektu może natomiast być widoczna również poza funkcją.

In [5]:
```
def suma(a, b):
    a += b
    return a
```

In [6]:
```
x = 5
s = suma(x, 5)
```

In [7]:
```
s
```

Out[7]:
```
10
```

In [8]:
```
x
```

Out[8]: 5

Wywołanie funkcji `suma` nie zmieniło wartości zmiennej `x`. Zmodyfikowana została wyłącznie lokalna zmienna `a` (parametr), która przestała istnieć po zakończeniu działania funkcji.

Sprawdź:

- Wizualizacja lokalnego przypisania do parametru na stronie Python Tutor

Uwaga! W przypadku zmiennych przechowujących wartości typów złożonych (np. listy, słowniki) funkcja również nie może zmienić wartości zmiennej w przypadku przekazania jej jako argumentu. Oznacza to, że po zakończeniu działania funkcji zmienna będzie wskazywać na tę samą listę czy słownik. Funkcja może natomiast zmienić zawartość samej listy lub słownika.

Sprawdź poniższe przykłady:

In [9]:
```
def nieudane_czyszczenie_listy(lista):
    lista = []
```

In [10]:
```
oceny = [1, 2, 3]
nieudane_czyszczenie_listy(oceny)
```

Funkcja nie zmodyfikowała wartości zmiennej `oceny`:

In [11]:
```
oceny
```

Out[11]: [1, 2, 3]

Sprawdź:

- Wizualizacja przypisania nowej listy do parametru na stronie Python Tutor

Kolejny przykład:

In [12]:
```
def zmien_pierwszy_element_listy(lista, wartosc):
    lista[0] = wartosc
```

In [13]:
```
oceny = [1, 2, 3, 4]
```

In [14]:
```
zmien_pierwszy_element_listy(oceny, 5)
```

In [15]:
```
oceny
```

Out[15]: [5, 2, 3, 4]

Sprawdź:

- Wizualizacja zmiany elementu przekazanej listy na stronie Python Tutor

Parametry z wartością domyślną

Jeżeli parametr ma wartość domyślną, to możemy jej nie podawać przy wywołaniu funkcji.

In [16]:
```
def cena_brutto(cena_netto, vat=0.23):
    return cena_netto * (1 + vat)
```

Wywołanie z wartościami dla wszystkich parametrów:

In [17]:
```
cena_brutto(200, 0.08)
```

Out[17]:
```
216.0
```

Brak wartości dla parametru `vat`. Użyta jest wartość domyślna, czyli `0.23`:

In [18]:
```
cena_brutto(100)
```

Out[18]:
```
123.0
```

Parametry z wartością domyślną nazywane są często parametrami **opcjonalnymi** (ich użycie w wywołaniu funkcji jest opcjonalne) w przeciwieństwie do pozostałych **obowiązkowych** parametrów.

Kolejny przykład: obliczenie długości wektora. Obowiązkowa jedna współrzędna, dwie opcjonalne (obliczenia w jednym, dwóch lub trzech wymiarach zrealizowane jedną funkcją):

In [19]:
```
def dlugosc_wektora(x, y=0.0, z=0.0):
    return (x**2 + y**2 + z**2) ** 0.5
```

In [20]:
```
dlugosc_wektora(3.0)
```

Out[20]:
```
3.0
```

In [21]:
```
dlugosc_wektora(3.0, 4.0)
```

Out[21]:
```
5.0
```

In [22]:
```
dlugosc_wektora(3.0, 4.0, 12.0)
```

Out[22]:
```
13.0
```

Błędy związane z nieprawidłowym przekazaniem argumentów do funkcji powodują zazwyczaj zgłoszenie wyjątku `TypeError`. W poniższym wywołaniu

nie podano obowiązkowej współrzędnej `x`, więc właśnie taki wyjątek zostanie zgłoszony:

In [23]:
```
dlugosc_wektora()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[23], line 1
----> 1 dlugosc_wektora()

TypeError: dlugosc_wektora() missing 1 required positional argument: 'x'
```

Parametry bez wartości domyślnej muszą poprzedzać te z wartością domyślną. Sama próba zdefiniowania poniższej funkcji, która nie przestrzega tej reguły, spowoduje wyjątek `SyntaxError`:

In [24]:
```
def cena_brutto(vat=0.23, cena_netto):
    return cena_netto * (1 + vat)
```

```
  Cell In[24], line 1
    def cena_brutto(vat=0.23, cena_netto):
                              ^
SyntaxError: parameter without a default follows parameter with a default
```

Kiedy obliczana jest wartość parametru z wartością domyślną?

Wartości parametrów domyślnych obliczane są raz — podczas **definiowania** funkcji, a nie podczas wykonania. Wypróbuj poniższy przykład:

In [25]:
```
def wyswietl_imie_i_nazwisko(imie, nazwisko=input("nazwisko: ")):
    print(f"Witaj {imie} {nazwisko}!")
```

```
nazwisko: Kowalski
```

In [26]:
```
wyswietl_imie_i_nazwisko("Jan", "Nowak")
```

```
Witaj Jan Nowak!
```

In [27]:
```
wyswietl_imie_i_nazwisko("Piotr")
```

```
Witaj Piotr Kowalski!
```

In [28]:
```
wyswietl_imie_i_nazwisko("Jan")
```

```
Witaj Jan Kowalski!
```

Przykład z pytaniem o nazwisko jest dość sztuczny, ale dobrze pokazuje, kiedy obliczana jest wartość domyślna. Dalej zobaczymy bardziej naturalny przykład i błąd, który może wystąpić przy użyciu listy jako wartości domyślnej.

Jeżeli chcemy, żeby wartość brakującego parametru była obliczana podczas wywołania funkcji, to zazwyczaj robi się to tak:

In [29]:
```
def wyswietl_imie_i_nazwisko(imie, nazwisko=None):
    if nazwisko is None:
        nazwisko = input("nazwisko: ")
    print(f"Witaj {imie} {nazwisko}!")
```

In [30]:
```
wyswietl_imie_i_nazwisko("Jan", "Nowak")
```

```
Witaj Jan Nowak!
```

In [31]:
```
wyswietl_imie_i_nazwisko("Piotr")
```

```
nazwisko: Kowalski
Witaj Piotr Kowalski!
```

In [32]:
```
wyswietl_imie_i_nazwisko("Anna")
```

```
nazwisko: Kowalski
Witaj Anna Kowalski!
```

Użycie list, słowników i innych modyfikowalnych obiektów jako domyślnych wartości parametrów

W związku z powyższym sposobem wyznaczania wartości parametrów z wartością domyślną trzeba bardzo uważać na obiekty mogące zmieniać swoją wartość (listy, słowniki, obiekty klas zdefiniowanych za pomocą konstrukcji `class`).

Przykład: funkcja dopisująca wynik (np. zawodów, eksperymentu) zadany pierwszym argumentem, do tabeli wyników zadanej drugim argumentem. Zwracana jest tabela z dopisanym wynikiem. Jeżeli tabela wyników nie została przekazana jako parametr, to zakładamy, że jest to pierwszy wynik i zwracana jest jednoelementowa lista.

In [33]:
```
def dopisz_wynik(wynik, tabela=[]):
    tabela.append(wynik)
    return tabela
```

Wywołanie z podaną tabelą wyników:

In [34]:
```
wyniki = [1, 2, 3]
```

In [35]:
```
dopisz_wynik(4, wyniki)
```

Out[35]:
```
[1, 2, 3, 4]
```

Wywołanie bez zadanej tabeli wyników:

In [36]:
```
dopisz_wynik(10)
```

Out[36]:
```
[10]
```

Wynik jest prawidłowy, ale spróbujmy jeszcze raz:

In [37]:
```
dopisz_wynik(20)
```

Out[37]:
```
[10, 20]
```

Domyślna wartość parametru `tabela` jest obliczana tylko raz — jest nią pusta lista utworzona podczas definicji funkcji. Kolejne wywołania używające wartości domyślnej nie tworzą nowej listy, tylko używają tej już utworzonej. Modyfikacja tej listy (dodanie elementu przez funkcję `dopisz_wynik`) jest widoczna podczas kolejnych wywołań próbujących używać tej wartości domyślnej.

Jeżeli chcemy, żeby przy każdym wywołaniu z użyciem wartości domyślnej tworzona była nowa lista, to możemy funkcję zdefiniować podobnie do funkcji `wyswietl_imie_i_nazwisko`:

In [38]:
```
def dopisz_wynik(wynik, tabela=None):
    if tabela is None:
        tabela = []
    tabela.append(wynik)
    return tabela
```

In [39]:
```
dopisz_wynik(5, [1, 2, 3, 4])
```

Out[39]:
```
[1, 2, 3, 4, 5]
```

In [40]:
```
dopisz_wynik(10)
```

Out[40]:
```
[10]
```

In [41]:
```
dopisz_wynik(5)
```

Out[41]:
```
[5]
```